

SciToolAgent: a knowledge-graph-driven scientific agent for multitool integration

Received: 9 December 2024

Accepted: 15 July 2025

Published online: 20 August 2025



Keyan Ding^{1,2,6}, Jing Yu^{2,3,6}, Junjie Huang², Yuchen Yang², Qiang Zhang^{1,2,4}✉ & Huajun Chen^{1,2,5}✉

Scientific research increasingly relies on specialized computational tools, yet effectively utilizing these tools requires substantial domain expertise. While large language models show promise in tool automation, they struggle to seamlessly integrate and orchestrate multiple tools for complex scientific workflows. Here we present SciToolAgent, a large language model-powered agent that automates hundreds of scientific tools across biology, chemistry and materials science. At its core, SciToolAgent leverages a scientific tool knowledge graph that enables intelligent tool selection and execution through graph-based retrieval-augmented generation. The agent also incorporates a comprehensive safety-checking module to ensure responsible and ethical tool usage. Extensive evaluations on a curated benchmark demonstrate that SciToolAgent outperforms existing approaches. Case studies in protein engineering, chemical reactivity prediction, chemical synthesis and metal–organic framework screening further demonstrate SciToolAgent’s capability to automate complex scientific workflows, making advanced research tools accessible to both experts and nonexperts.

Scientific research increasingly depends on specialized computational tools that automate essential tasks ranging from data analysis to result visualization. While these tools have become indispensable for advancing scientific discovery, their growing complexity and diversity create substantial barriers to adoption. For example, chemists routinely use specific tools for molecular simulation, property prediction and compound design. Beginner researchers may lack the technical expertise required to effectively utilize these powerful resources, potentially impeding scientific progress. To address this challenge, a promising approach involves leveraging artificial intelligence technologies to efficiently organize and orchestrate the use of scientific tools^{1–3}.

Large language models (LLMs), as cutting-edge artificial intelligence methods, have demonstrated unprecedented capabilities across diverse domains, from natural language understanding to complex reasoning tasks^{4–6}. Recent research has demonstrated promising advances in integrating LLMs with domain-specific scientific tools^{7,8}.

In chemistry, several pioneering systems—including Coscientist⁹, ChemChat¹⁰, ChemCrow¹¹ and CACTUS¹²—have enabled autonomous chemical research through LLM-tool integration. Similar progress has emerged in biological sciences, where systems such as GeneGPT¹³, CRISPR-GPT¹⁴, GenoAgent¹⁵ and ProtAgents¹⁶ have enhanced LLMs for specialized tasks, including RNA sequencing, gene editing and protein discovery. The application of tool-augmented LLMs has further expanded to materials discovery (LLMatDesign¹⁷ and ChatMOF¹⁸), electronic design (ChatEDA¹⁹) and mechanics engineering (MechAgents²⁰). These systems typically leverage in-context learning within established frameworks such as ReAct²¹, which combines LLM-based reasoning with tool execution. However, current approaches face two critical limitations: (1) they operate with a restricted set of tools (typically fewer than 20), limiting their broader applicability, and (2) they often overlook crucial safety and ethical considerations in scientific research²².

¹College of Computer Science and Technology, Zhejiang University, Hangzhou, China. ²Zhejiang Key Laboratory of Intelligent Manufacturing for Functional Chemicals, ZJU-Hangzhou Global Scientific and Technological Innovation Center, Zhejiang University, Hangzhou, China. ³The Polytechnic Institute, Zhejiang University, Hangzhou, China. ⁴ZJU-UIUC Institute, Zhejiang University, Haining, China. ⁵State Key Laboratory of Ocean Sensing, Hangzhou, China. ⁶These authors contributed equally: Keyan Ding, Jing Yu. ✉e-mail: qiang.zhang.cs@zju.edu.cn; huajunsir@zju.edu.cn

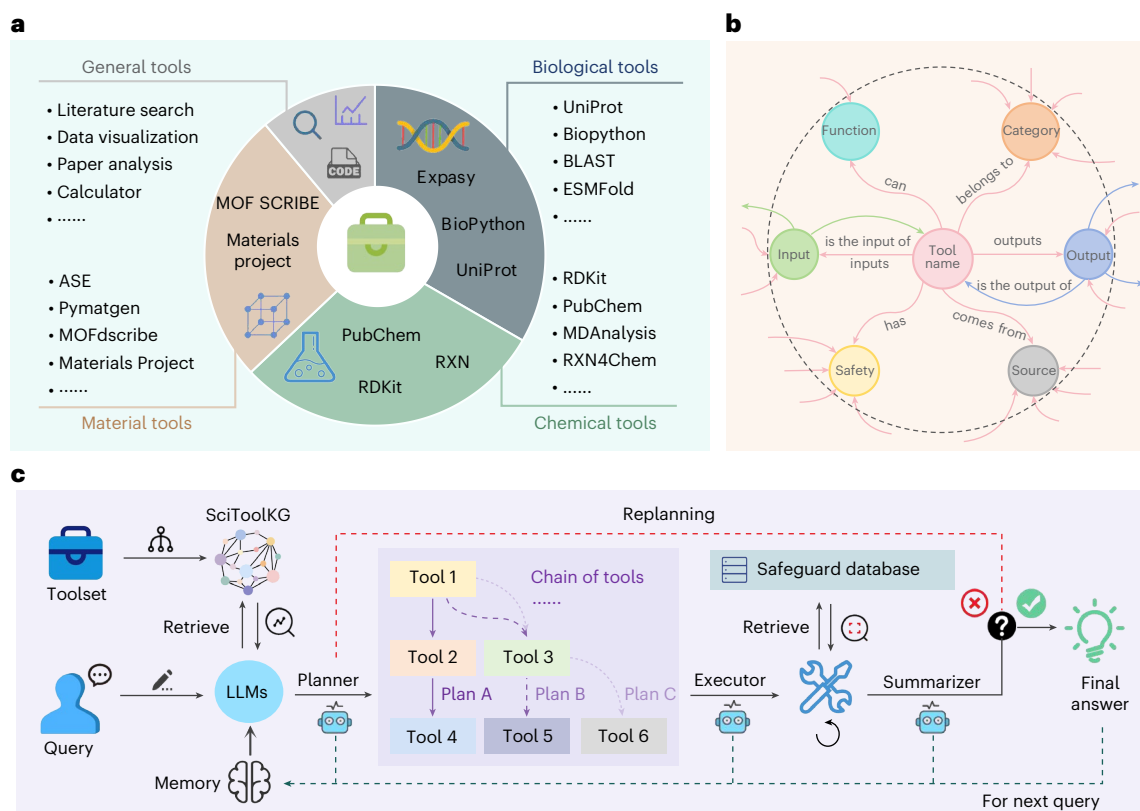


Fig. 1 | Overview of SciToolAgent. **a**, The toolset involved in our study, which includes general tools as well as frequently used biology, chemistry and material-related tools. ASE, atomic simulation environment. **b**, The schema of SciToolKG, encompassing diverse information about each tool such as input/output formats, specific functionalities, safety levels and so on. **c**, The overall workflow of SciToolAgent. Given a user query, an LLM-based Planner generates a chain of tools using a SciToolKG-based retrieve-augmented generation,

followed by sequential implementation through an LLM-based Executor.

A safety check module is then conducted to ensure ethical and safe solutions by retrieving a safeguard database. Finally, an LLM-based Summarizer concludes the implementation results, assesses the problem-solving process and prompts the Planner to generate a new chain of tools if necessary. The final answers will be sent to the memory module as context for the next query.

Current agent frameworks that rely on naive in-context learning often struggle with complex scientific problems due to their inability to account for intrinsic dependencies among a wide array of tools^{23,24}. These dependencies are predominantly characterized by sequential relationships, where the output of one tool serves as the input for the next, necessitating a precise operational order. The failure to account for these interdependent relationships frequently leads to suboptimal solutions and reduced efficiency when handling multistep scientific workflows.

In this study, we present SciToolAgent, an agent framework that effectively integrates extensive and diverse scientific tools with LLMs (Fig. 1). Specifically, SciToolAgent utilizes advanced LLMs as a Planner, Executor and Summarizer to autonomously plan, execute multiple tools and summarize the solution for scientific tasks. SciToolAgent introduces two key innovations: a comprehensive scientific tool knowledge graph (SciToolKG) that encodes relationships among hundreds of tools across biology, chemistry and materials science, and an integrated safety module that ensures responsible scientific research. The SciToolKG enables informed tool selection and combination by explicitly modeling tool dependencies, prerequisites and compatibility. Our safety module continuously monitors the execution pipeline to prevent potentially harmful outcomes, addressing a critical limitation in existing frameworks that often overlook the ethical implications of automated scientific discovery.

We evaluate SciToolAgent using our scientific tool evaluation (SciToolEval) benchmark, comprising 531 diverse scientific problems that span multiple domains and complexity levels. Quantitative

analysis demonstrates that SciToolAgent achieves an overall accuracy of 94%, surpassing state-of-the-art baselines by 10%. We further validate SciToolAgent's effectiveness through case studies across four scenarios: protein design and analysis, chemical reactivity prediction, chemical synthesis and analysis, and metal-organic framework (MOF) material screening. These studies showcase SciToolAgent's capability to autonomously orchestrate complex multitool workflows while maintaining solution reliability and accuracy.

Results

Overview of SciToolAgent

SciToolAgent is an LLM-based agent designed to overcome the limitations of existing systems in scientific tool integration. While traditional frameworks often fail due to their naive in-context learning approaches that overlook tool interdependencies, SciToolAgent addresses this challenge through a scientific tool knowledge graph (SciToolKG) that mediates between LLMs and scientific tools. The toolset involved in SciToolAgent includes general tools such as search engines and code interpreters, as well as specialized scientific tools in biology, chemistry and materials domains (Fig. 1a). To establish the intricate relationships among extensive tools, we manually construct a SciToolKG (Fig. 1b), encompassing diverse information about each tool. This SciToolKG plays a crucial role in the planning process, enabling LLMs to make well-informed decisions regarding tool selection and combination for optimal problem-solving. The implementation workflow of SciToolAgent is illustrated in Fig. 1c, which consists of three main components: a tool Planner, a tool Executor and a solution

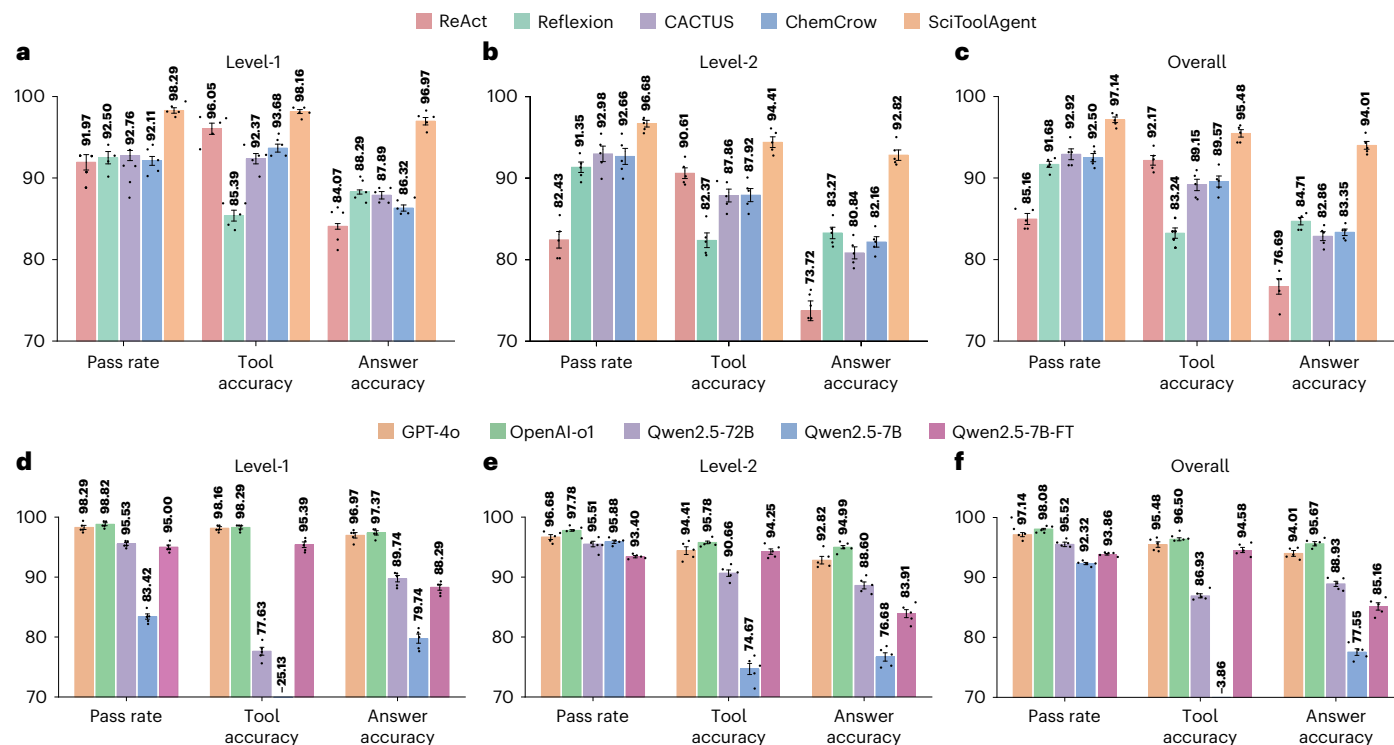


Fig. 2 | Comparison results of different agents and foundation models. **a–c**, The performance of different agents on SciToolEval, including Level-1 (**a**) and Level-2 (**b**) and the overall performance (**c**). The proposed SciToolAgent consistently outperforms the baseline approaches across all evaluation metrics and levels. **d–f**, The performance of different foundation models within

SciToolAgent, including Level-1 (**d**) and Level-2 (**e**) and the overall performance (**f**). The OpenAI-o1 model achieves the best result, while the GPT-4o reaches an optimal trade-off between accuracy and cost. Data are presented as mean values. The number of experimental replicates is $n = 5$, and the error bars are the standard error of the mean.

Summarizer, all of which are powered by LLMs. The Planner utilizes retrieve-augmented generation with SciToolKG to generate a chain of tools that can solve the given query. The Executor implements these tools sequentially and retries if errors occur. A retrieve-based safety check module is used to identify the potential harmful responses within tools. The Summarizer compiles and synthesizes outputs from various tools to generate the final answer. In case of failure to solve the problem using the current plan, as judged by the summarizer, it will prompt the Planner to refine the chain of tools.

Performance on SciToolEval

Dataset: To quantitatively evaluate the performance of SciToolAgent, we curated SciToolEval, a comprehensive scientific tool evaluation dataset, comprising 531 diverse scientific questions across various fields, including molecular property prediction, protein analysis and material retrieval. The process of dataset construction is elaborated in ‘Construction of SciToolEval’ section in the Methods. The final dataset is divided into two levels: Level-1 includes 152 questions that can be addressed using a single tool, while Level-2 comprises 379 questions that require the use of multiple tools.

Evaluation metrics: To rigorously assess the effectiveness of SciToolAgent, we utilize three distinct evaluation metrics: (1) pass rate, which quantifies the proportion of successfully completed queries; (2) tool planning accuracy, which measures the alignment between the tool selection and sequencing generated by the agent and a reference plan verified by human experts; and (3) final answer accuracy, which compares the final solution generated by agents with the standard answer, assessing the correctness of the output. For both tool planning accuracy and final answer accuracy, we prompt GPT-4o to perform a similarity evaluation between the generated response and the ground truth.

Baselines: We compare SciToolAgent’s performance against two state-of-the-art LLM-based tool agents including ReAct²¹, Reflexion²⁵,

CACTUS¹² and ChemCrow¹¹. The ReAct framework involves an iterative process where LLMs generate a reasoning step (that is, thought) to determine how to tackle the problem and select a tool (that is, action) to take, followed by a result analysis (that is, observation) through LLMs. The Reflexion framework enhances ReAct by incorporating a feedback mechanism into the decision-making process, which includes self-evaluation, self-reflection and memory components. ChemCrow and CACTUS are ReAct-based agents with domain-specific tools and templates for chemical synthesis and material science, respectively. While their default toolsets are limited, we adapt them with our full toolset for a fair comparison.

Foundation models: In our experiments, we utilize different foundation models for the Planner, Executor and Summarizer, including the proprietary model (OpenAI’s GPT-4o (default) and o1) as well as the state-of-the-art open-source model (Qwen2.5-72B). All of these models are accessed via official application programming interfaces (APIs) with a standard configuration. In addition, we explore Qwen2.5-7B, an open-source LLM with a smaller parameter scale but higher efficiency. Given its reduced inference capacity, we fine-tuned it using specific instructions for tool learning. This setup allows us to evaluate the cost-effectiveness of SciToolAgent across models of varying capacities, ensuring that our framework can perform effectively while accommodating resource-constrained environments.

Result analysis: Figure 2a–c shows the performance of five GPT-4o-based agents (ReAct, Reflexion, ChemCrow, CACTUS and SciToolAgent) across different levels on SciToolEval. SciToolAgent outperformed the other agents in all evaluated metrics. For the task of using a single tool (Level-1), the agent needs to select an appropriate tool from the candidate tools, execute it correctly and summarize the answer. For the task of using multiple tools (Level-2), the agent requires additional capabilities, including selecting multiple tools, generating a suitable plan and executing them in a specific sequence.

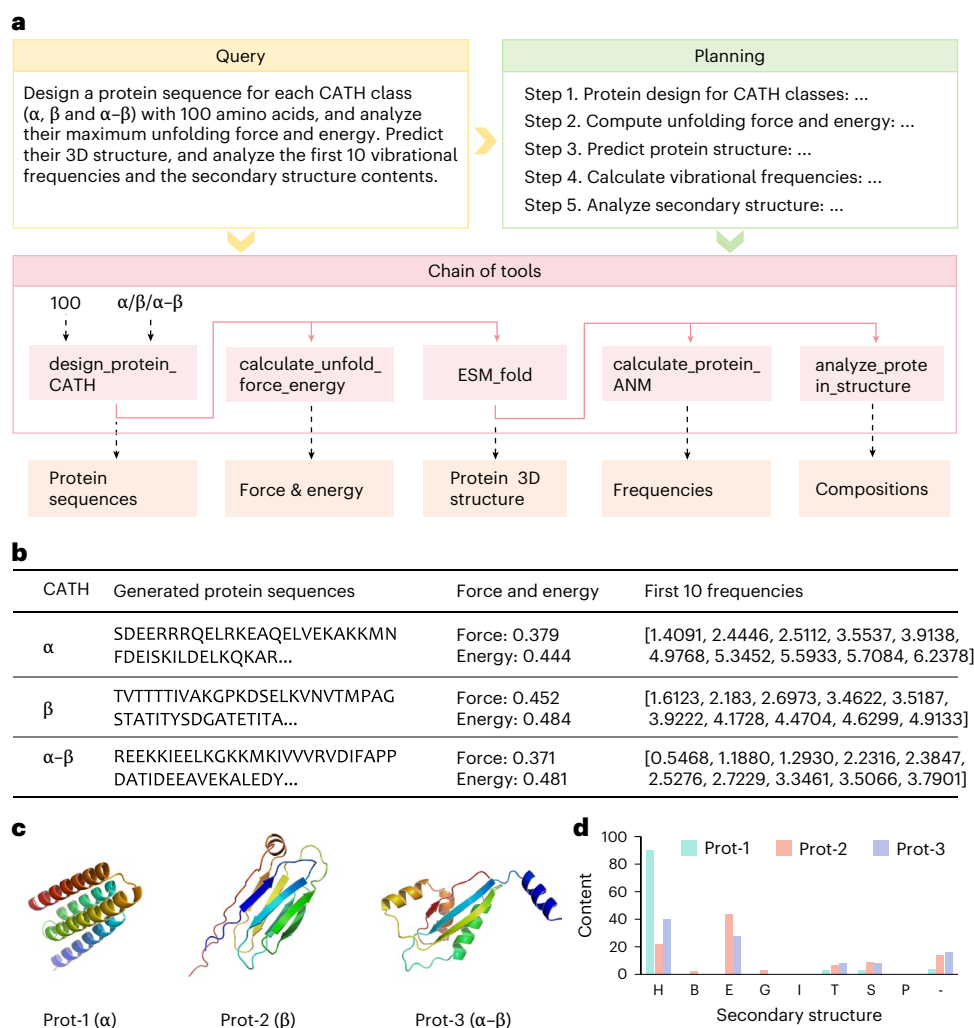


Fig. 3 | Workflow and results of protein design and analysis using SciToolAgent. a, An overview of the input query, tool planning, chain of tools and corresponding outcomes. CATH is a protein structure classification database. **b**, The generated protein sequences, the corresponding unfolding

forces and energies, and the first ten vibrational frequencies for each class. **c**, The predicted three-dimensional (3D) structures of the generated protein sequences. **d**, The fractional content of secondary structure elements.

SciToolAgent demonstrates a more substantial advantage in solving multitool problems, outperforming ReAct and Reflexion by approximately 20% and 10%, respectively, in terms of final answer accuracy. Compared with ReAct-based agents ChemCrow and CACTUS, SciToolAgent also achieves an absolute gain of 10–12% on both levels. Empirically, we observe that ReAct and Reflexion exhibit limitations in tool planning and execution when confronted with complex multitool tasks and a large set of candidate tools. Both approaches lack a comprehensive, global task-planning strategy. Although Reflexion uses a feedback mechanism to improve the pass rate and accuracy, its extensive trial-and-error process results in a lower tool planning accuracy. SciToolAgent utilizes SciToolKG, largely enhancing the accuracy of tool planning. Owing to the introduction of the chain of tools, SciToolAgent can plan step-by-step tasks from a global perspective, reducing trial-and-error costs and improving its planning proficiency. During tool execution, the chain of tools helps the model to focus on the current subtask, avoiding interference from irrelevant information, thus enhancing execution accuracy.

Figure 2d–f shows the performance of five different foundation models within the SciToolAgent framework. OpenAI o1 consistently outperforms other models across all metrics in both Level-1 and Level-2 tasks, followed by GPT-4o, which achieves slightly lower accuracy but maintains strong overall performance. Qwen2.5-72B performs

competitively but demonstrates a slight drop in tool planning accuracy and final answer accuracy. By contrast, Qwen2.5-7B shows comparatively lower performance, especially in tool planning, indicating limitations in its capacity to manage complex tool integration. However, a fine-tuned version of Qwen2.5-7B (FT), trained using data generated from SciToolKG, exhibits notable improvement (+10%) and achieves results closer to those of the higher-capacity models (Qwen2.5-72B). Notably, GPT-4o achieves an optimal trade-off between accuracy and cost, delivering near-top performance across all metrics while maintaining a lower API cost compared with o1. This makes GPT-4o the default foundation model in our implementation of SciToolAgent. More experimental results are presented in Supplementary Section 3.

Case studies

To further validate the utility of SciToolAgent, we conduct four real-world scientific research tasks with SciToolAgent: (1) protein design and analysis, (2) chemical reactivity prediction, (3) chemical synthesis and analysis and (4) MOF materials screening. More case analyses and comparisons are reported in Supplementary Section 5.

Protein design and analysis. Protein design is crucial for advancing biological fields such as drug discovery, enzyme engineering and synthetic biology. Designing proteins with desired properties is a complex task

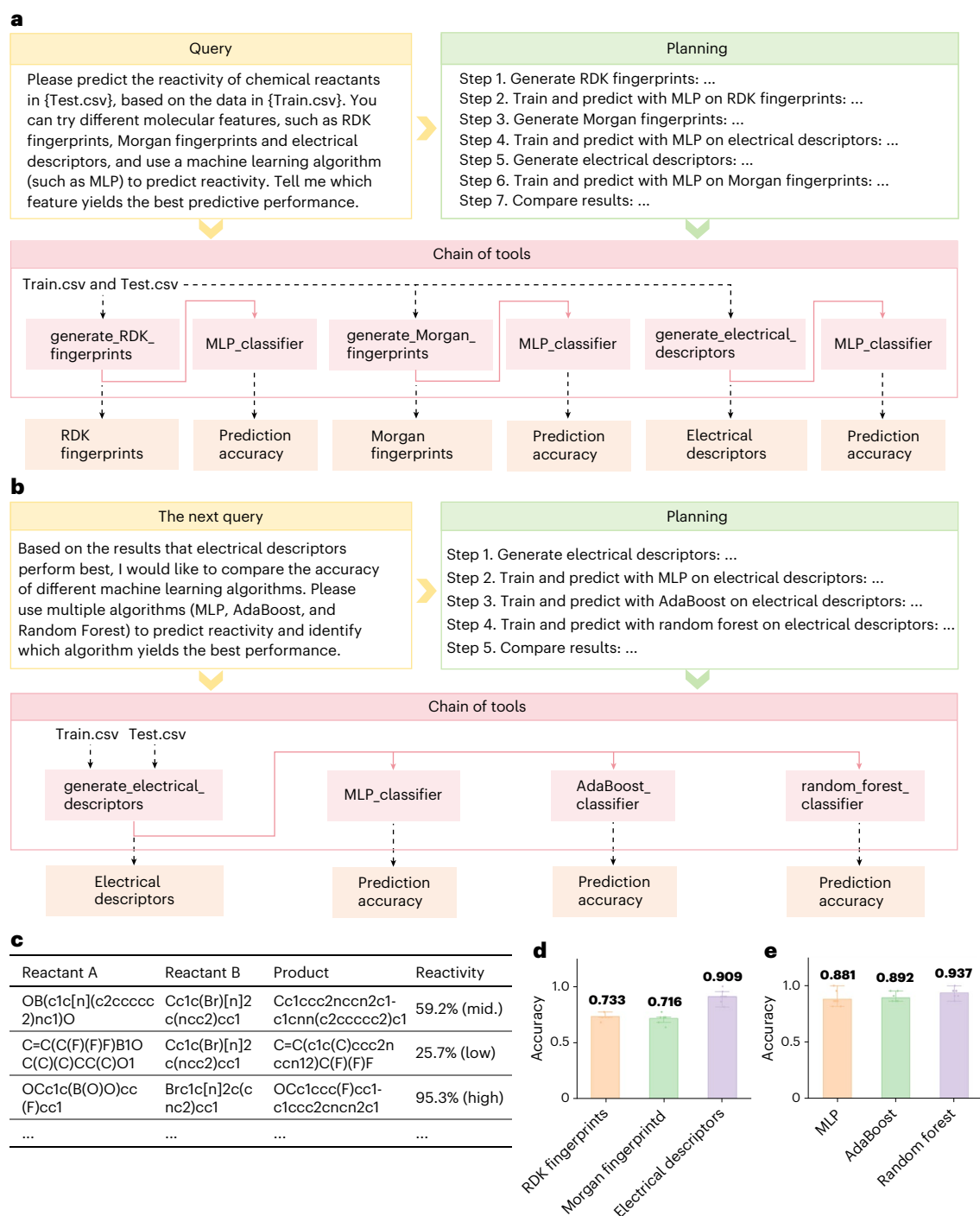


Fig. 4 | Workflow and results of machine learning-based chemical reactivity prediction using SciToolAgent. a, An overview of the input query, tool planning, chain of tools and corresponding outcomes. **b,** An overview of the next query, tool planning, chain of tools and corresponding outcomes. **c,** The dataset used

for training and testing. **d,** The prediction accuracy with different molecular features. **e,** The prediction accuracy with different machine learning algorithms. Data are presented as mean values. The number of experimental replicates is $n = 8$, and the error bars are the standard error of the mean.

that typically requires the integration of multiple bioinformatics tools for sequence generation, folding prediction and structural analysis. Traditional workflows necessitate substantial expertise to navigate and operate the diverse set of specialized tools effectively. SciToolAgent streamlines this process by autonomously orchestrating the necessary tools to achieve comprehensive protein design and analysis.

In this case, we illustrate how SciToolAgent manages a multistep workflow for protein design. Given a user query (Fig. 3a), the process begins with protein sequence generation, using Chroma²⁶ to create

sequences based on predefined secondary structure types (for instance, α -helix, β -sheet or mixed). Next, unfolding force and energy calculations are performed using ProteinForceGPT¹⁶ to assess the mechanical stability of the generated sequences. The protein structure is then predicted using ESMFold²⁷, followed by vibrational frequency computation with the anisotropic network model (ANM)²⁸ implemented in ProDy²⁹ to evaluate the dynamic stability of the structure. Finally, secondary structure analysis is carried out using the Dictionary of Secondary Structure of Proteins (DSSP) module from BioPython³⁰

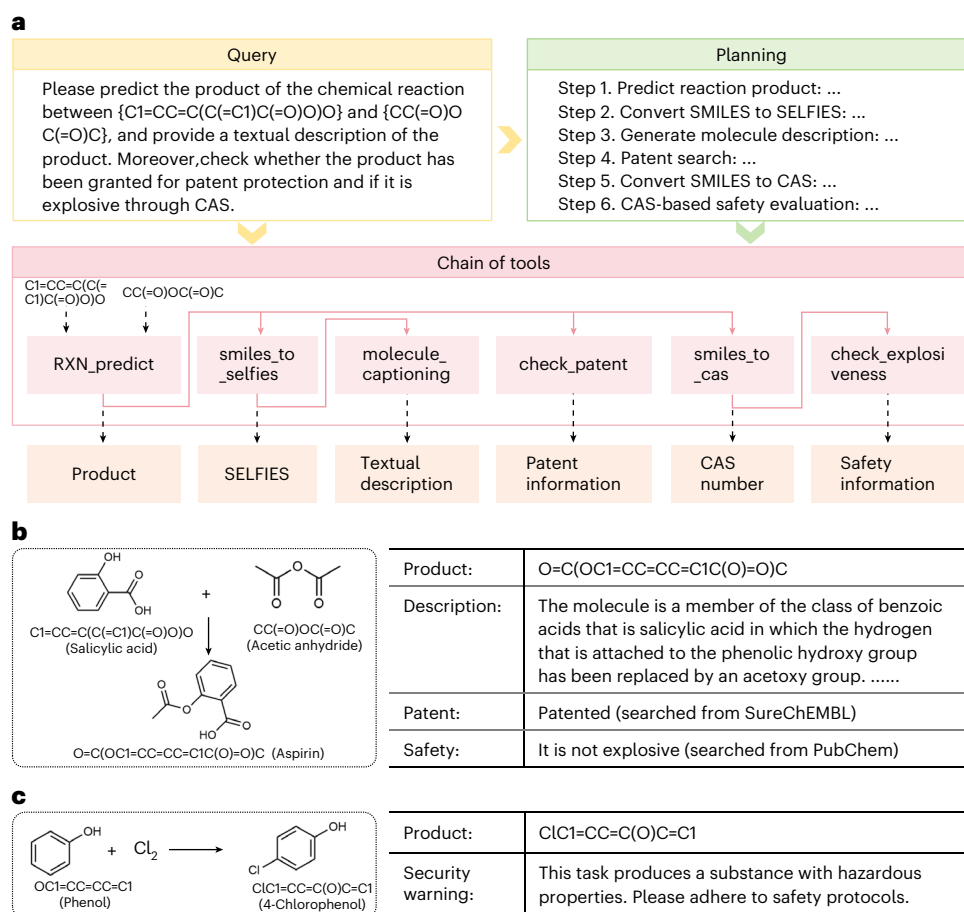


Fig. 5 | Workflow and results of chemical synthesis and analysis using SciToolAgent. a, An overview of the input query, tool planning, chain of tools and corresponding outcomes. **b**, Results of the chemical reaction between salicylic acid and acetic anhydride, including the product and the corresponding

description, patent and safety information. **c**, Results of the chemical reaction between phenol and chlorine, issuing a security warning due to the product's toxicity.

to confirm the structural elements. The results (Fig. 3b,c) from this workflow demonstrate the capability of SciToolAgent to autonomously orchestrate multiple tools and produce reliable outcomes. By contrast, the baseline methods ReAct and Reflexion failed to address this task effectively due to their incorrect or missing tools (see Supplementary Section 4 for details).

Chemical reactivity prediction. Predicting chemical reactivity is a critical aspect of drug design and organic synthesis. Accurate predictions of how chemical compounds will react with one another can accelerate the development of new compounds and reduce the need for labor-intensive experimental processes. In this case, we use SciToolAgent to predict the reactivity of amide condensation reactions, a crucial class of reactions in organic chemistry. Specifically, we aim to predict the reactivity of various chemical substrates based on molecular features and identify the most effective machine learning algorithms for this task. The dataset (Fig. 4c) used for this study comprises different combinations of reactants along with the corresponding reaction products, and the conversion rate of the product, quantified by the liquid chromatography area percent based on ultraviolet absorbance at 254 nm. The goal is to classify the reactivity of these substrate combinations into three categories: low, medium and high activity.

The workflow (Fig. 4a) begins with an initial query prompting SciToolAgent to predict reactivity using various molecular features, such as RDKit fingerprints (an RDKit topological fingerprint), Morgan fingerprints and electrical descriptors, and to test a machine learning algorithm, such as the multilayer perceptron (MLP), to determine

which feature provides the best predictive performance. SciToolAgent plans the steps accordingly, first generating each type of fingerprint and descriptor, then training and testing the MLP classifier on each feature set. The results (Fig. 4d) indicate that electrical descriptors outperform the other features in predicting reactivity. Based on this finding, a follow-up query asks SciToolAgent to further explore the performance of different machine learning algorithms, specifically comparing MLP, AdaBoost and random forest classifiers, using only electrical descriptors as input features (Fig. 4b). SciToolAgent then generates a new plan to test each algorithm sequentially on this feature set, evaluating their predictive accuracy in classifying reactivity. By comparing the outcomes, SciToolAgent determines which algorithm is most effective with the electrical descriptors. The results (Fig. 4e) reveal that the random forest algorithm yields the highest prediction accuracy among the tested algorithms, making it the optimal choice for this task. Notably, ReAct encountered tool redundancy and Reflexion encountered hallucinations in completing this task (see Supplementary Section 4 for details).

Chemical synthesis and analysis. Chemical synthesis is a cornerstone of chemistry, pivotal for the design and creation of novel compounds, particularly in pharmaceutical research. Accurately predicting reaction outcomes and understanding the properties of synthesized molecules are crucial steps for efficient and safe drug design. In this case, we demonstrate how SciToolAgent automates a synthesis and analysis pipeline, encompassing reaction prediction, product characterization, intellectual property assessment and safety evaluation.

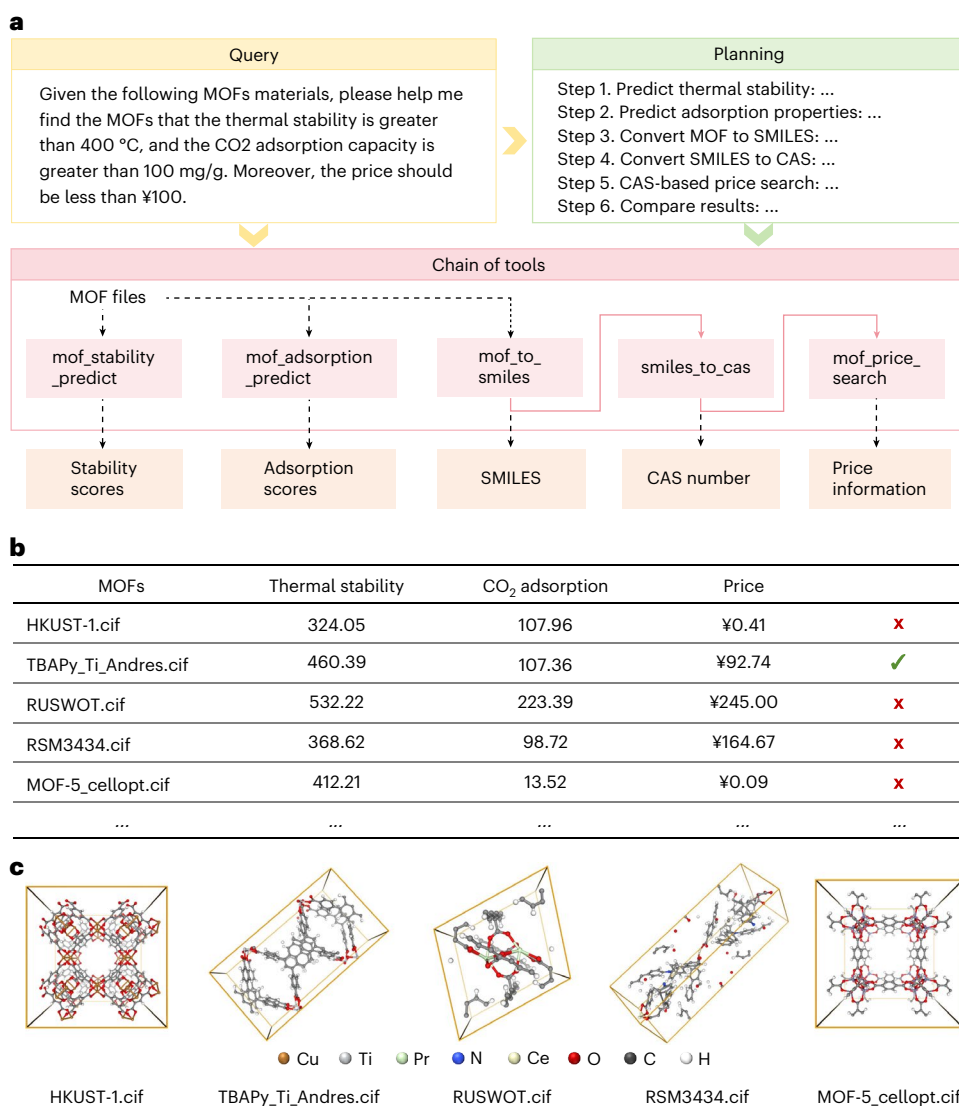


Fig. 6 | Workflow and results of MOF materials screening using SciToolAgent.

a, An overview of the input query, tool planning, chain of tools and corresponding outcomes. **b**, A summary of the obtained results showing the thermal

stability, CO₂ adsorption capacity and market price for each MOF. The green checkmarks indicate MOFs that meet all specified criteria. **c**, A visualization of these MOF structures.

The first task involves predicting and analyzing the outcome of an esterification reaction between salicylic acid and acetic anhydride. SciToolAgent autonomously plans and executes this multistep process by orchestrating the necessary tools in sequence, as shown in Fig. 5a. Specifically, SciToolAgent first uses the RXN-for-chemistry³¹ tool to predict the reaction product (denoted by SMILES). Following this, the agent converts the SMILES notation of the product into the SELFIES format for compatibility and uses a molecule captioning tool (BioT5+³²) to generate a textual description. Next, the agent conducts a patent search using a patent check tool across the SureChEMBL database³³. Finally, for explosiveness assessment, the agent converts the SMILES notation to a CAS number and retrieves relevant safety information from PubChem³⁴. The obtained results are summarized in Fig. 5b.

Furthermore, we conducted a similar query for another reaction: the chlorination of phenol, where phenol reacts with chlorine (Fig. 5c). SciToolAgent follows a comparable workflow for this query, beginning with reaction prediction. However, the internal safety check module in SciToolAgent identifies the product (4-chlorophenol) as a hazardous compound, and issues a security warning indicating that the resulting product is toxic and requires careful handling and specific safety precautions. This built-in safety check is crucial, as it alerts researchers

to potential risks early in the workflow, helping to prevent accidental exposure to toxic substances and ensuring that safety protocols are adhered to throughout the experimental process. Conversely, neither ReAct nor Reflexion incorporated safety checks during the synthesis process, which led to substantial risks in chemical experimentation. While both methods were able to predict reaction outcomes, they failed to identify the toxic nature of the product in the chlorination of phenol case (see Supplementary Section 4 for details).

MOF materials screening. MOFs are a class of crystalline porous materials with versatile applications in areas such as gas storage, catalysis, drug delivery and separation processes. The identification of optimal MOFs for specific scenarios often involves screening materials on the basis of multiple performance criteria, such as high thermal stability and efficient adsorption capacity. In this case, we demonstrate how SciToolAgent can streamline the MOF screening process by automating the selection and analysis of MOFs based on predefined criteria.

Given a batch of MOFs from MOFDB³⁵, the workflow (Fig. 6a) begins with a user query that prompts SciToolAgent to find MOFs with thermal stability above 400 °C, CO₂ adsorption capacity over

100 mg g⁻¹ and a price under ¥100. In response, SciToolAgent generates a plan and executes a chain of tools to filter MOFs on the basis of these criteria. Specifically, the agent first assesses the thermal stability of each MOF by using a machine learning-based predictive model (MOFSimplify³⁶) that calculates thermal stability scores. Next, the agent predicts their CO₂ adsorption capacities using a molecular simulation software (RASPA2 (ref. 37)). Last, the agent inquires about their pricing information. As the pricing retrieval tool requires CAS numbers as input, it first converts the structural data of MOFs (provided as CIF files) into SMILES format and subsequently translates these SMILES strings into CAS numbers. With the obtained CAS numbers, SciToolAgent accesses the commercial chemical database to retrieve market price information.

Figure 6b summarizes the partial results obtained from this automated screening workflow. For instance, by filtering MOFs on the basis of thermal stability, CO₂ adsorption capacity and price, SciToolAgent successfully identified TBAPy_Ti_Andres.cif as a highly promising candidate that met all predefined criteria. The visualization of selected MOF structures (Fig. 6c) further supports the analysis by highlighting structural elements contributing to stability and adsorption properties, offering valuable insights for subsequent experimental validation. By contrast, ReAct encountered input errors and Reflexion encountered hallucinations when attempting to address this task (see Supplementary Section 4 for details).

Discussion

SciToolAgent is an LLM-powered agent designed to address the challenges of integrating extensive scientific tools for advanced research across multiple domains. The primary advantage of SciToolAgent lies in its integration of diverse scientific tools through SciToolKG, which captures the intricacies of tool dependencies, input/output formats and application contexts. Previous LLM-based frameworks have been restricted by their limited toolsets and simple task-planning strategies. By contrast, SciToolAgent can dynamically create a chain of tools tailored to the specific requirements of each scientific task. This capability allows researchers to delegate repetitive or computationally intensive steps to SciToolAgent, making scientific inquiry more accessible and efficient for both domain experts and nonexperts.

One potential limitation of SciToolAgent lies in the manual construction of the SciToolKG knowledge graph. Although SciToolKG captures relationships and dependencies effectively, its scalability is constrained by the effort required to curate and update tool information. Automated approaches, such as using knowledge extraction techniques from scientific literature or metadata from tool documentation, could further enhance the scalability and granularity of SciToolKG. In addition, expanding the knowledge graph to incorporate emerging tools and resources will be essential as scientific research continues to evolve. To facilitate extensibility, we provide standardized APIs and templates to support third-party tool integration with minimal effort. Future versions will also include graphical user interface-based registration to reduce the barrier for domain experts without programming experience.

Another challenge lies in the reliance on the underlying LLM's capabilities to perform effectively across varied scientific tasks. While proprietary models such as GPT-4o demonstrate strong performance, they may be financially and technically inaccessible for all researchers, especially in resource-limited settings. Our experiments with open-source models indicate that fine-tuning with domain-specific data can enhance performance and partially bridge the gap between open-source and proprietary alternatives. However, even with fine-tuning, Qwen2.5-7B-FT still lags behind GPT-4o, especially in complex tool planning and multistep reasoning.

While challenges related to the scalability of SciToolKG and the reliance on proprietary LLM models remain, SciToolAgent offers a robust foundation for automating complex scientific workflows.

Future work will focus on automating the knowledge graph's maintenance, integrating more tools and enhancing open-source LLM capabilities to further democratize access to advanced scientific research. Ultimately, SciToolAgent showcases the potential of LLM-driven agents to streamline and empower scientific discovery, making sophisticated tools accessible to a broader audience.

Methods

This section presents the methodology of SciToolAgent, including the collection of scientific tools, the construction of SciToolKG and SciToolEval based on these tools, and the implementation details of Planner, Executor and Summarizer, as well as the LLMs behind them.

Collection of scientific tools

The collection of scientific tools for SciToolAgent follows a systematic process aimed at assembling a comprehensive, domain-specific and functionally diverse toolset. Initially, we identified key scientific domains that would benefit most from LLM integration, including biology, chemistry and materials science. We then curated a list of tools used frequently in these domains. The current toolset in SciToolAgent includes over 500 tools, spanning a wide range of functionalities. For instance, in biology, we included sequence alignment tools such as BLAST³⁸, protein structure prediction models such as ESMFold²⁷ and gene editing software such as CRISPR-related tools. In chemistry, our toolset features molecular dynamics simulations, cheminformatics libraries such as RDKit³⁹ and compound databases such as PubChem³⁴. For materials science, we integrated tools for crystal structure prediction, materials property databases and MOF property predictors such as MOFSimplify³⁶. More details of the tools are presented in Supplementary Section 1.

Construction of SciToolKG

SciToolKG serves as the backbone of SciToolAgent, providing a structured representation of the relationships, dependencies and operational details of a vast array of scientific tools. Formally, SciToolKG is represented as a directed graph $G = (V, E)$, where V represents a set of entities (nodes) and E represents a set of relations (edges) between these entities. Specifically, the tool nodes represent individual scientific tools, and the attribute nodes represent various tool properties and metadata. The edges capture the semantic relationships between the tool node and the attribute node.

The construction of SciToolKG followed a three-step process: tool characterization, schema development and graph population. The initial phase involved defining a set of attributes for each tool, including its purpose, specific functionalities, input/output formats, categories, sources and safety levels. These attributes were derived mainly from tool documentation. Then, a hierarchical schema was developed to model these attributes logically, including nodes representing tools and their properties and edges denoting relationships such as functional dependencies. The final stage involved populating the knowledge graph by encoding tool-specific metadata into a structured graph with triplets. In particular, relationships between tools—such as shared input/output formats or sequential task dependencies—were explicitly captured, creating a rich, interconnected graph. To ensure accuracy and usability, the knowledge graph underwent iterative refinement through expert validation.

Construction of SciToolEval

To evaluate SciToolAgent's performance, we constructed a comprehensive scientific tool evaluation (SciToolEval) dataset that encompasses a diverse array of real-world scientific tasks. This dataset serves as the first testing ground that can quantitatively assess agents' capabilities across various scientific domains. The process of constructing SciToolEval involves the following steps.

(1) Tool selection and question generation: The first step is selecting the appropriate tools and generating meaningful questions.

For single tool calling scenarios, we include a detailed description of the tool in the prompt and instruct GPT-4o to generate relevant questions. This ensures that the generated questions are directly actionable and pertinent to the tool's functionality.

(2) Tool execution and answer generation: Once the questions are generated, we proceed to tool execution and answer generation by utilizing the ReACT framework in conjunction with GPT-4o. ReACT²¹ has demonstrated its effectiveness in handling a limited number of tools. Therefore, only the tools relevant to the question are fed to ReACT, improving the accuracy of tool invocation.

(3) Manual review: The final step involves manual review to ensure the quality of generated answers. We invite three domain-specific experts to meticulously review the questions, tool usage and answers to verify their correctness and significance.

Finally, SciToolEval consists of 531 questions associated with the required tools and their corresponding answers. These questions cover single and multiple tools for problem-solving, and they are categorized into different levels based on difficulty: Level-1 involves simple information queries using a single tool; Level-2 includes sequential and parallel use of multiple tools, requiring basic planning, reasoning and summarization skills. More details of the dataset construction are presented in Supplementary Section 2.

Implementation of SciToolAgent

SciToolAgent's architecture revolves around three core components powered by LLMs: Planner, Executor and Summarizer (Fig. 1c). Each component plays a distinct role in the problem-solving workflow by leveraging the strengths of LLMs.

Planner. The Planner is responsible for devising a strategy to solve the given query. It leverages LLMs to interpret the question, query the SciToolKG and generate a plan of the chain of tools. Specifically, a retrieve-augmented generation approach based on the SciToolKG is proposed to identify and sequence the appropriate tools needed. The process begins with the LLM querying the SciToolKG based on the input question, followed by the steps described below.

(1) Full-graph retrieval: We first retrieve the most relevant k tools for the given question in the full graph of SciToolKG. This is achieved by calculating the semantic similarity between the question and all of the triples involved in tool functions within the SciToolKG. The similarity score helps in identifying tools that are most likely to be useful for the given query. Mathematically, let T_i represent a tool in the SciToolKG $\mathcal{G}$ and $S(q, T_i)$ denote the semantic similarity score between the question q and the tool T_i . This retrieval process can be formulated as

$$\mathbb{T}_{\text{full}} = \text{top-}k\{T_i | S(q, T_i), \forall T_i \in \mathcal{G}\}. \quad (1)$$

(2) Subgraph exploration: Once the initial set of tools is retrieved, we explore all subgraphs (that is, d -hop neighborhoods) of these tools in SciToolKG. The purpose of this step is to identify any additional tools that might be required in conjunction with the initially retrieved tools to solve the query. Mathematically, for each retrieved tool T_i , we calculate the semantic similarity between q and the tools $T_i \oplus T_j$, where $\oplus$ denotes semantic concatenation that combines the textual metadata of T_i and T_j . Here, T_j is the tool in the d -hop neighborhoods of T_i . The top- k tools with the highest similarity are obtained by

$$\mathbb{T}_{\text{sub}} = \text{top-}k\{T_j | S(q, T_i \oplus T_j), \forall T_j \in \mathcal{G}_{\text{sub}}^{T_i}\}. \quad (2)$$

(3) Tool combination and ranking: By using full-graph retrieval and subgraph exploration for each query, we obtain a total of k^2 tools. To optimize the utilization of these tools, we sort them according to the combined similarity score $S' = S(q, T_i) \times S(q, T_i \oplus T_j)$, prioritizing tools that are not only relevant to the question but also complementary

to each other. We then select the top- n tools ($n \leq k^2$) based on the combined similarity:

$$\mathbb{T}_{\text{comb}} = \text{top-}n\{T_i | S(q, T_i) \times S(q, T_i \oplus T_j), \forall T_j \in \mathcal{G}_{\text{sub}}^{T_i}, \forall T_i \in \mathcal{G}\}. \quad (3)$$

(4) Chain-of-tools generation: Based on the selected tools and the neighborhood information from SciToolKG, we prompt LLMs to generate a chain of tools that outlines the tools required to solve the query. This chain is optimized to ensure that the tools are used in the most effective order, leveraging their dependencies and functionalities. The final chain of tools can be represented as

$$\mathbb{T}_{\text{chain}} = \{T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_m\} \in \mathbb{T}_{\text{comb}}. \quad (4)$$

Unless otherwise specified, the parameters for retrieval are set as follows: $d = 3$, $k = 5$ and $n = 10$. For measuring semantic similarity S , we calculate the cosine distance between two text embedding vectors obtained from a pretrained embedding model.

Executor. The Executor aims to ensure the effective implementation of the planned chain of tools. This component is designed to handle tool inputs, manage execution processes and address any errors that may arise during execution. It also integrates a robust safety module to monitor and control potentially hazardous inputs or outputs. The execution process involves the four steps as follows.

(1) Input preparation: The Executor uses LLMs to extract the required input parameters for the current tool in the chain. This involves parsing the question, understanding the context and identifying the necessary data to feed into the tool. Inputs are formatted according to the specifications of the tool recorded in SciToolKG. This may include converting data types, structuring the input in a specific format or integrating multiple inputs.

(2) Tool execution: The Executor invokes the tool with the prepared inputs, which involves interfacing with the tool's API. The execution process is actively monitored to track progress, capture outputs and detect any anomalies or errors. The Executor logs execution details to ensure traceability. Once the tool has been executed, the outputs are captured and further processed as needed to be compatible with the subsequent tools in the chain.

(3) Error handling and retries: The Executor is equipped with protocols to detect errors during tool execution. Errors can occur due to incorrect inputs, tool malfunctions or other unforeseen issues. When an error is detected, the Executor adjusts the inputs on the basis of predefined rules or heuristics to rectify the issue.

(4) Retrieve-based safety check module: In parallel with the execution process, the Executor incorporates a retrieve-based safety check module. This module evaluates each step's inputs and outputs against a set of predefined safety criteria to prevent the generation of harmful substances or unethical use of scientific tools. To this end, we first construct a comprehensive safeguard database that includes hazardous compounds collected from PubChem and toxic proteins collected from UniProtKB. By comparing the outcomes with the substances contained in this database, we can assess the potential harmfulness of molecules or protein sequences generated during execution.

Specifically, given a molecule x , the safety check module calculates the feature similarity between x and all entries in the safeguard database D . The similarity is quantified using the average of the Tanimoto coefficient, Dice coefficient and Cosine coefficient, each measuring the degree of similarity between molecular fingerprints⁴⁰:

$$\hat{S}_{\text{mol}}(x, D) = \max_{y_i \in D} \frac{1}{3} (\text{Tanimoto}(x, y_i) + \text{Dice}(x, y_i) + \text{Cosine}(x, y_i)). \quad (5)$$

For proteins, we use the Smith–Waterman algorithm⁴¹ for pairwise local alignment between the given sequence and entries in the

safeguard database, and the percentage of matching serves as the similarity score:

$$\hat{S}_{\text{prot}}(x, D) = \max_{y_i \in D} \text{Smith-Waterman}(x, y_i). \quad (6)$$

If $\hat{S}(x, D)$ exceeds a certain threshold $\delta = 0.95$, indicating a close to known hazardous or toxic entities, the safety module flags this output as potentially dangerous. To ensure the efficiency of execution, only the tools marked with high risks in SciToolKG are required to pass through the safety check module.

Summarizer. After execution, the Summarizer compiles and synthesizes outputs from various tools to generate the final answer, ensuring coherence and accuracy. It also assesses the success of the problem-solving process and, if necessary, prompts the Planner to refine the chain of tools for optimal results. The implementation of the Summarizer involves several key steps.

(1) Synthesis of outputs: The Summarizer collects the outputs from all the tools executed in the chain and integrates them into a cohesive response. This involves (1) combining the outputs from different tools and ensuring that all relevant information is included, (2) verifying that the outputs from different tools align with each other and do not contain conflicting information and (3) structuring the final answer in a logical and easy-to-understand format.

(2) Iterative refinement: If the initial chain of tools fails to produce satisfactory results, as determined by the Summarizer, it prompts the Planner to refine the plan. This refinement involves identifying reasons for any failures or suboptimal outcomes, suggesting modifications to the chain of tools (such as adding, removing, reordering or retrieving tools) and repeating the execution process with the refined chain of tools to achieve improved results.

Foundation models. The foundation models behind the Planner, Executor and Summarizer in SciToolAgent are LLMs. Our experiments have tried different LLMs, including the well-known proprietary model (OpenAI's GPT-4o) as well as the state-of-the-art open-source models (Qwen2.5-72B). Moreover, we fine-tuned Qwen2.5-7B, an open-source LLM with a smaller parameter scale but higher efficiency, using specific instructions for tool learning and deployed it locally on a graphics processing unit server.

(1) Instruction generation: To enhance the performance of Qwen2.5-7B, we generate specific instructions tailored to scientific tool learning. These instructions guide the model in understanding the functionalities of various tools and their appropriate usage. The instruction generation process is similar to the construction of the SciToolEval dataset, utilizing SciToolKG and GPT-4o for automated generation (encompassing tool selection, question generation, tool execution and answer generation). This approach yields a large number of instructions, although they are not manually reviewed.

(2) Fine-tuning LLMs with instructions: We fine-tuned Qwen2.5-7B using LoRA (low-rank adaptation⁴²), leveraging the constructed instruction dataset to enhance the model's capacity for tool-specific planning and execution. Separate fine-tuning was conducted for the Planner, Executor and Summarizer, with task-specific instructions ensuring the optimization of each component's functionality. The training loss is measured using the next token prediction loss⁴³, defined as

$$L = - \sum_{t=1}^T \log P(y_t | y_{<t}, X; \theta), \quad (7)$$

where y_t represents the target token at time step t , X is the input text and θ denotes the model parameters. This fine-tuning process effectively enhances the instruction following the capability of Qwen2.5-7B, making it a cost-effective choice for implementing the Planner, Executor and Summarizer.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

The toxic compound data were obtained from PubChem (<https://pubchem.ncbi.nlm.nih.gov/#query=toxic&tab=compound>), and the toxic protein data were sourced from UniProtKB (<https://www.uniprot.org/uniprotkb?query=toxic&facets=reviewed%3Atrue>). The SciToolEval dataset is available via Zenodo at <https://doi.org/10.5281/zenodo.15691499> (ref. 44). Source data are provided with this paper.

Code availability

The source code of SciToolAgent is available via GitHub at <https://github.com/hicai-zju/scitoolagent> and via Zenodo at <https://doi.org/10.5281/zenodo.15707181> (ref. 45).

References

- Birhane, A., Kasirzadeh, A., Leslie, D. & Wachter, S. Science in the age of large language models. *Nat. Rev. Phys.* **5**, 277–280 (2023).
- Schick, T. et al. Toolformer: language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* **36**, 68539–68551 (2023).
- Yang, R. et al. GPT4Tools: teaching large language model to use tools via self-instruction. *Adv. Neural Inf. Process. Syst.* **36**, 71995–72007 (2024).
- Guo, T. et al. What can large language models do in chemistry? A comprehensive benchmark on eight tasks. *Adv. Neural Inf. Process. Syst.* **36**, 59662–59688 (2023).
- Zhao, W. X. et al. A survey of large language models. Preprint at <https://arxiv.org/abs/2303.18223> (2023).
- Min, B. et al. Recent advances in natural language processing via large pre-trained language models: a survey. *ACM Comput. Surveys* **56**, 1–40 (2023).
- Wang, L. et al. A survey on large language model based autonomous agents. *Front. Comput. Sci.* **18**, 186345 (2024).
- Ramos, M. C., Collison, C. J. & White, A. D. A review of large language models and autonomous agents in chemistry. *Chem. Sci.* **16**, 2514–2572 (2025).
- Boiko, D. A., MacKnight, R., Kline, B. & Gomes, G. Autonomous chemical research with large language models. *Nature* **624**, 570–578 (2023).
- Janakaraman, N., Erdmann, T., Swaminathan, S., Laino, T. & Born, J. Language models in molecular discovery. In *Drug Development Supported by Informatics* (eds Satoh, H. et al.) 121–141 (Springer, 2024).
- Bran, A. M. et al. Augmenting large language models with chemistry tools. *Nat. Mach. Intell.* **6**, 525–535 (2024).
- McNaughton, A. D. et al. CACTUS: chemistry agent connecting tool usage to science. *ACS Omega* **9**, 46563–46573 (2024).
- Jin, Q., Yang, Y., Chen, Q. & Lu, Z. GeneGPT: augmenting large language models with domain tools for improved access to biomedical information. *Bioinformatics* **40**, btad075 (2024).
- Huang, K. et al. CRISPR-GPT: an LLM agent for automated design of gene-editing experiments. Preprint at <https://arxiv.org/abs/2404.18021> (2024).
- Liu, H. & Wang, H. GenoTEX: a benchmark for evaluating LLM-based exploration of gene expression data in alignment with bioinformaticians. Preprint at <https://arxiv.org/abs/2406.15341> (2024).
- Ghafarirollahi, A. & Buehler, M. J. ProtAgents: protein discovery via large language model multi-agent collaborations combining physics and machine learning. *Digital Discovery* **3**, 1389–1409 (2024).

17. Jia, S., Zhang, C. & Fung, V. LLMatDesign: autonomous materials discovery with large language models. Preprint at <https://arxiv.org/abs/2406.13163> (2024).
18. Kang, Y. & Kim, J. ChatMOF: an artificial intelligence system for predicting and generating metal–organic frameworks using large language models. *Nat. Commun.* **15**, 4705 (2024).
19. Wu, H. et al. ChatEDA: a large language model powered autonomous agent for EDA. *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.* **43**, 3184–3197 (2024).
20. Ni, B. & Buehler, M. J. MechAgents: large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mech. Lett.* **67**, 102131 (2024).
21. Yao, S. et al. ReAct: synergizing reasoning and acting in language models. In *International Conference on Learning Representations* <https://iclr.cc/virtual/2023/oral/12647> (2023).
22. He, J. et al. Control risk for potential misuse of artificial intelligence in science. Preprint at <https://arxiv.org/abs/2312.06632> (2023).
23. Liu, X. et al. ToolNet: connecting large language models with massive tools via tool graph. Preprint at <https://arxiv.org/abs/2403.00839> (2024).
24. Hao, S., Liu, T., Wang, Z. & Hu, Z. ToolkenGPT: augmenting frozen language models with massive tools via tool embeddings. *Adv. Neural Inf. Process. Syst.* **36**, 45870–45894 (2024).
25. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. & Yao, S. Reflexion: language agents with verbal reinforcement learning. *Adv. Neural Inf. Process. Syst.* **36**, 8634–8652 (2024).
26. Ingraham, J. B. et al. Illuminating protein space with a programmable generative model. *Nature* **623**, 1070–1078 (2023).
27. Lin, Z. et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* **379**, 1123–1130 (2023).
28. Atilgan, A. R. et al. Anisotropy of fluctuation dynamics of proteins with an elastic network model. *Biophys. J.* **80**, 505–515 (2001).
29. Bakan, A., Meireles, L. M. & Bahar, I. Prody: protein dynamics inferred from theory and experiments. *Bioinformatics* **27**, 1575–1577 (2011).
30. Cock, P. J. et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics* **25**, 1422 (2009).
31. Schwaller, P. et al. Molecular transformer: a model for uncertainty-calibrated chemical reaction prediction. *ACS Central Science* **5**, 1572–1583 (2019).
32. Pei, Q. et al. BioT5+: towards generalized biological understanding with IUPAC integration and multi-task tuning. In *Findings of the Association for Computational Linguistics: ACL 2024*, 1216–1240 (Association for Computational Linguistics, 2024).
33. Papadatos, G. et al. SureChEMBL: a large-scale, chemically annotated patent document database. *Nucleic Acids Res.* **44**, D1220–D1228 (2016).
34. Kim, S. et al. Pubchem substance and compound databases. *Nucleic Acids Res.* **44**, D1202–D1213 (2016).
35. Bobbitt, N. S. et al. MOFX-DB: an online database of computational adsorption data for nanoporous materials. *J. Chem. Eng. Data* **68**, 483–498 (2023).
36. Nandy, A. et al. Mofsimply, machine learning models with extracted stability data of three thousand metal–organic frameworks. *Sci. Data* **9**, 74 (2022).
37. Dubbeldam, D., Calero, S., Ellis, D. E. & Snurr, R. Q. RASPA: molecular simulation software for adsorption and diffusion in flexible nanoporous materials. *Molecular Simulation* **42**, 81–101 (2016).
38. BLAST: basic local alignment search tool. *NIH* <https://blast.ncbi.nlm.nih.gov> (2024).
39. RDKit: open-source cheminformatics software. *RDKit* <http://www.rdkit.org> (2024).
40. Bajusz, D., Rácz, A. & Héberger, K. Why is tanimoto index an appropriate choice for fingerprint-based similarity calculations? *J. Cheminformatics* **7**, 1–13 (2015).
41. Smith, T. F. et al. Identification of common molecular subsequences. *J. Mol. Biol.* **147**, 195–197 (1981).
42. Hu, E. J. et al. LoRA: low-rank adaptation of large language models. In *International Conference on Learning Representations* <https://iclr.cc/virtual/2022/poster/6319> (2022).
43. Vaswani, A. et al. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, 5998–6008 (2017).
44. Yu, J. Dataset for the paper “SciToolAgent: A knowledge graph-driven scientific agent for multi-tool integration”. *Zenodo* <https://doi.org/10.5281/zenodo.15691499> (2025).
45. Yu, J. & Ding, K. HICAI-ZJU/SciToolAgent: V1.0.1. *Zenodo* <https://doi.org/10.5281/zenodo.15707182> (2025).

Acknowledgements

This work is funded by National Science and Technology Major Project (2023ZD0120802, Q.Z.), Zhejiang Provincial ‘Jianbing’ ‘Lingyan’ Research and Development Program of China (2025C01097, K.D. and Q.Z.), NSFC62301480 (K.D.), NSFC62302433 (Q.Z.), NSFCU23A20496 (Q.Z.), NSFCU23B2055 (H.C.), the Fundamental Research Funds for the Central Universities (226-2023-00138, H.C.), Zhejiang Provincial Natural Science Foundation of China (LQ24F020007, Q.Z.) and Hangzhou West Lake Pearl Project Leading Innovative Youth Team Project (TD2023017, K.D.). The artificial-intelligence-driven experiments, simulations and model training were performed on the robotic AI-Scientist platform of the Chinese Academy of Sciences.

Author contributions

K.D. conceived the study and designed the method. J.Y. and J.H. implemented the method and experiments. K.D. and J.Y. performed the result analyses. Y.Y. developed the online service. K.D., J.Y., Q.Z. and H.C. wrote and revised the paper. Q.Z. and H.C. supervised the entire project. All authors wrote the paper, reviewed it and approved the final paper.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s43588-025-00849-y>.

Correspondence and requests for materials should be addressed to Qiang Zhang or Huajun Chen.

Peer review information *Nature Computational Science* thanks Jihan Kim, Lei Shi and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. Primary Handling Editor: Kaitlin McCardle, in collaboration with the *Nature Computational Science* team.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature America, Inc. 2025

Reporting Summary

Nature Portfolio wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Portfolio policies, see our [Editorial Policies](#) and the [Editorial Policy Checklist](#).

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a	Confirmed
☐	☒ The exact sample size (<i>n</i>) for each experimental group/condition, given as a discrete number and unit of measurement
☐	☒ A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
☒	☐ The statistical test(s) used AND whether they are one- or two-sided <i>Only common tests should be described solely by name; describe more complex techniques in the Methods section.</i>
☒	☐ A description of all covariates tested
☒	☐ A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
☐	☒ A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
☒	☐ For null hypothesis testing, the test statistic (e.g. <i>F</i> , <i>t</i> , <i>r</i>) with confidence intervals, effect sizes, degrees of freedom and <i>P</i> value noted <i>Give P values as exact values whenever suitable.</i>
☒	☐ For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
☒	☐ For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
☒	☐ Estimates of effect sizes (e.g. Cohen's <i>d</i> , Pearson's <i>r</i>), indicating how they were calculated

Our web collection on [statistics for biologists](#) contains articles on many of the points above.

Software and code

Policy information about [availability of computer code](#)

Data collection	We used open-source software for data collection, including python== 3.10, beautifulsoup4==4.12.3, html2text==2020.1.16, biopython==1.84 and openai==1.55.0. Detailed package dependencies and versions are documented in our code repository: https://github.com/HICAI-ZJU/SciToolAgent .
Data analysis	We used open-source software for data analysis, including numpy==2.1.3, scipy==1.14.1, rdkit==2024.3.3, langchain==0.3.8, aiofiles==24.1.0, aiohttp==3.9.5, fastapi==0.115.5 and pandas==2.2.3. Detailed package dependencies and versions are documented in our code repository: https://github.com/HICAI-ZU/SciToolAgent .

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors and reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Portfolio [guidelines for submitting code & software](#) for further information.

Data

Policy information about [availability of data](#)

All manuscripts must include a [data availability statement](#). This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A description of any restrictions on data availability
- For clinical datasets or third party data, please ensure that the statement adheres to our [policy](#)

The toxic compound data were obtained from PubChem (<https://pubchem.ncbi.nlm.nih.gov>), and the toxic protein data were sourced from UniProtKB (<https://www.uniprot.org>). The SciToolEval dataset can be found at <https://doi.org/10.5281/zenodo.15691499> without any restrictions.

Research involving human participants, their data, or biological material

Policy information about studies with [human participants or human data](#). See also policy information about [sex, gender \(identity/presentation\), and sexual orientation](#) and [race, ethnicity and racism](#).

Reporting on sex and gender	N/A (This study does not involve human participants)
Reporting on race, ethnicity, or other socially relevant groupings	N/A
Population characteristics	N/A
Recruitment	N/A
Ethics oversight	N/A

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☒ Life sciences ☐ Behavioural & social sciences ☐ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see [nature.com/documents/nr-reporting-summary-flat.pdf](https://www.nature.com/documents/nr-reporting-summary-flat.pdf)

Life sciences study design

All studies must disclose on these points even when the disclosure is negative.

Sample size	We conducted the experiment with 5 independent runs to account for the inherent variability in large language model outputs. The average result over these 5 runs was reported. This sample size was chosen as a practical balance between computational cost and result stability, and it was found to be sufficient to produce consistent performance trends across our evaluations.
Data exclusions	No data exclusions.
Replication	All codes necessary for replication are available on GitHub at https://github.com/hicai-zju/scitoolagent .
Randomization	The randomization process was applied in the study.
Blinding	This study does not involve participant groups; therefore, blinding is not applicable.

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

n/a	Involved in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology and archaeology
☒	☐ Animals and other organisms
☒	☐ Clinical data
☒	☐ Dual use research of concern
☒	☐ Plants

Methods

n/a	Involved in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging

Plants

Seed stocks

Report on the source of all seed stocks or other plant material used. If applicable, state the seed stock centre and catalogue number. If plant specimens were collected from the field, describe the collection location, date and sampling procedures.

Novel plant genotypes

Describe the methods by which all novel plant genotypes were produced. This includes those generated by transgenic approaches, gene editing, chemical/radiation-based mutagenesis and hybridization. For transgenic lines, describe the transformation method, the number of independent lines analyzed and the generation upon which experiments were performed. For gene-edited lines, describe the editor used, the endogenous sequence targeted for editing, the targeting guide RNA sequence (if applicable) and how the editor was applied.

Authentication

Describe any authentication procedures for each seed stock used or novel genotype generated. Describe any experiments used to assess the effect of a mutation and, where applicable, how potential secondary effects (e.g. second site T-DNA insertions, mosaicism, off-target gene editing) were examined.